
Modular Programming – Practical Work

Week 9

GUI

Notations: W1Q1 = Week 1, Questions of type 1

Question types:

- Type 1: theory, concepts *knowledge*
- Type 2: understanding typically based on code reading *understanding*
- Type 3: producing yourself code *know-how*

A moodle deposit space is open every week for you to upload your solutions. Solutions of the exercises are provided one week later.

Important points:

- 602_F and 602_D classes should upload solutions **before Monday 24h00 the following week**
- **2 or 3 practical work will be corrected** and will count for one quarter of the intermediate test note

Conventions about fonts:

- Lucida Console is used for Java reserved commands
- Courier is used for the code
- Arial is used for the text

W9Q1 – 1

1. Cite 2 differences between JComboBox and JList.
 - a. With a JComboBox we can edit the text in the header field, lists are not editable
 - b. The list of the JComboBox is hidden until we click on it
 - c. With a JList we can do multiple selections (if programmed to do so)
 - d. A JComboBox includes a default scrollbar not a JList
2. We have seen during the course that a JComboBox can generate events of type ItemEvent.
 - What is the other type of event that a JComboBox can generate?
 - What is the difference between these 2 types of events?

Remark: Use the Javadoc to provide your answer. Don't describe the « low-level » events (focus, key, mouse events, ...).

The other type of event is an **ActionEvent**.

An event of type **ActionEvent** is generated when the user selects an item in the list (or when he enters a text on the keyboard and presses enter)

An event **ItemEvent** is generated when the state of one item of the **ComboBox** is changing. When a user selects a new item in the **JComboBox**, the previous selected item is deselected.
3. We have seen in the course that it is possible to create an instance of an inner class (internal class) to manage the events from the GUI (the condition for the inner class is to implement the adequate Listener interface). Cite 2 other ways to implement the management of events.
 - a. Using an **internal anonymous class** that implements the ad-hoc interface **Listener**
 - b. Using an internal class that extends an **“adapter” class** (for example: **MouseAdapter**)
 - c. Implement the ad-hoc **Listener interface** in the main class (cf. exercise **W9Q2**)

W9Q2-1

Analyze the code below, then:

1. Explain:
 - a. What are the events that are generated when the user presses on the key 'A'.
event key pressed: A
event key typed: a
event key released: A
 - b. What are the events that are generated when the user presses the combination of keys 'Maj + a' ('shift + a').
event key pressed: Maj
event key pressed: A
event key typed: A
event key released: A
event key released: Maj
2. The author of the code did not use inner classes (internal classes) for the management of the events. The author preferred to implement directly the interface `KeyListener` in the main class. Would it have been possible to obtain the same result with an inner class (internal class)

Yes

```

import java.awt.Color;
import java.awt.event.KeyListener;
import java.awt.event.KeyEvent;
import javax.swing.JFrame;
import javax.swing.JTextArea;

public class KeyboardSimpleGUI extends JFrame implements
KeyListener
{
    private String line = ""; // line of textarea
    private JTextArea textArea; // textarea to display output

    // constructor
    public KeyboardSimpleGUI()
    {
        super( "A Very Simple Type Writer" );

        textArea = new JTextArea( 10, 15 ); // set up JTextArea
        textArea.setText( "Press any key on the keyboard..." );

        textArea.setEnabled( false ); // disable textarea
        textArea.setDisabledTextColor( Color.BLACK );
        // set text color
        add( textArea ); // add textarea to JFrame

        addKeyListener( this );
        // allow frame to process key events
    } // end KeyboardSimpleGUI constructor

    // handle press of any key
    public void keyPressed( KeyEvent event )
    {
        System.out.printf("event key pressed: %s\n",
            KeyEvent.getKeyText( event.getKeyCode() ) );
        // output pressed key
    } // end method keyPressed

    // handle release of any key
    public void keyReleased( KeyEvent event )
    {
        System.out.printf("event key released: %s\n",
            KeyEvent.getKeyText( event.getKeyCode() ) );
        // output released key
    } // end method keyReleased

    // handle press of an action key
    public void keyTyped( KeyEvent event )
    {
        System.out.printf("event key typed: %s\n",
            event.getKeyChar() );
        line = line + String.format("%s", event.getKeyChar());
        textArea.setText(line);
    } // end method keyTyped

    public static void main( String args[] )
    {
        KeyboardSimpleGUI simpleTypeWriter = new
        KeyboardSimpleGUI();
        simpleTypeWriter.setDefaultCloseOperation(
        JFrame.EXIT_ON_CLOSE );
        simpleTypeWriter.setSize( 350, 400 ); // set frame size
        simpleTypeWriter.setVisible( true ); // display frame
    } // end main

} // end class

```